

Skanda Nagendra HSC Software Engineering

Major Project 2026 Report – SEN

Table of Contents

Table of Contents.....	1
Requirements & Specifications.....	5
App Requirements.....	5
Feasibility Study.....	6
Gantt Chart.....	7
Task.....	7
Week.....	7
Task.....	8
Week.....	8
Analysis Phase.....	9
Pseudocode.....	9
Context Diagram & Data Flow Diagram.....	12
Structure Chart (Login & Quiz).....	13
Creating the Database.....	13
Design Phase.....	16
Progression within the App.....	16
Signs & Quizzes in order:.....	16

Development Phase.....	18
The Database.....	18
The Flask App Routing.....	19
Account Features [Login & Registration].....	20
Data Dictionary.....	21
User Table in Database.....	21
Question Table in Database.....	22
QuizResult Table in Database.....	23
Marks Table in Database.....	24
Signs.....	24
AssignedCourses Table in Database.....	25
Session Variables.....	25
Other Variables used within the python script.....	26
Methods/Functions in app.py.....	27
App Routes.....	29
Integration Phase.....	31
Running the Server Locally.....	31
The HTML Pages.....	32
Images/Icons & Stylesheet.....	32
The Database.....	33
Testing & Debugging Phase.....	33
Testing Cycles.....	33
Installation Phase.....	36
Maintenance Phase.....	36

Company Statement.....	36
END-USER LICENSE AGREEMENT (EULA).....	37
SEN — Light Guard Studios.....	37
IMPORTANT — READ CAREFULLY:.....	37
1. Grant of License.....	37
2. Ownership.....	37
3. Restrictions.....	38
4. Educational and Development Use.....	38
5. Termination.....	38
6. Disclaimer of Warranty.....	39
7. Limitation of Liability.....	39
8. Governing Law.....	39
9. Entire Agreement.....	39
10. Contact Information.....	40
Journal.....	41
Term 3 Holidays [28/09/2025 — 12/10/2025].....	41
17/10/2025.....	41
28/10/2025.....	41
29/10/2025 - 19/12/2025.....	41
20/12/2025 - 31/1/2026.....	42
2/2/2026.....	42
3/2/2026.....	42
5/2/2026 - 24/2/2026.....	42
25/2/2026.....	42

28/2/2026 - 19/3/2026.....	43
Post-Term 1 Holidays.....	43
24/4/2026 - 30/4/2026.....	43
1/5/2026.....	43
2/5/2026 - 6/5/2026.....	44
7/5/2026.....	44
8/5/2026 - 13/5/2026.....	44
14/5/2026 - 31/5/2026.....	44
1/6/2026 - 4/6/2026.....	44
5/6/2026 - 7/6/2026.....	45
8/6/2026 - 10/6/2026.....	45
11/6/2026.....	45
12/6/2026 - 15/6/2026.....	45
16/6/2026 - 18/6/2026.....	46
19/6/2026 - 21/6/2026.....	46
22/6/2026 - 24/6/2026.....	46
Client Meeting Logs.....	47
1. 28/10/2025.....	47
2. 30/4/2025.....	47
3. 11/6/2026.....	48
4. 19/6/2026.....	49
Example Question Bank.....	50

Requirements & Specifications

- Deepens the understanding of students who are to learn Software Engineering
- Tests students' knowledge on the subtopics that come up in the Preliminary course.

App Requirements

- There are to be different access levels, as follows, with specific privileges:
 - Guest
 - Level when you are accessing the account, but are not logged in.
 - Can see what the app has to offer, cannot undergo any of the tests or see any questions.
 - Can Log In / Sign Up
 - Student
 - The main focus. Has logged in with a school [student] email [Format: lettersAndNumbersOnly@education.nsw.gov.au].
 - Can only do the things below if they have been assigned to a teacher, and students cannot be assigned to more than one teacher.
 - Can take tests. Results of said tests can be accessed only by their assigned teacher.
 - Can read 'signs' which are articles of information regarding the content they will learn about.
 - Teacher
 - Is logged in with a school [teacher] email. [Format: lettersAndNumbersOnly@det.nsw.edu.au].
 - Can be assigned students, see their names and test scores, and can remove students from their list.
 - Can read signs and allow students to access specific courses.
 - Can send a report or appeal to an admin.
 - Admin

- Can only gain this position by being promoted from teacher.
- Can see the names of teachers and students only. Can receive reports or appeals from teachers only.
- Can ban accounts.
- Can add questions to the question bank, after being approved by the Host.
- Can send messages
- Host
 - Can promote or demote teacher accounts to or from admin. This position should only be given to trusted accounts.
 - Can add questions to the question bank.

Feasibility Study

- The Project is to be in the format of a PWA, meaning it needs to have a website aspect and use service workers to ensure offline usage capabilities.
- Currently, it seems feasible to create a question bank for one area first, looking at the momentum of the project, but if a lot gets finished quickly, the other two areas may also be included.
- Though not a lot is known about how this project can be made, there is reason to believe the software that is to be employed will be able to meet the previously stated expectations.

Gantt Chart

[illegible]

Come up with a timeline for what is to be presented in order for the project to have progression, continuity and allow users to be able to follow along with what is going on														
Make an overview of a small portion of the project and receive feedback. This overview will include placeholders in UI.														
Start working on coding navigation between screens and basic interactivity with a small portion of the project.														
Integrate some of the more finished project art designs & potentially music.														

[illegible]

<i>Receive feedback on what has been done, improve certain parts accordingly before proceeding</i>														
<i>Code the progression of remaining sections of the project, adjusting some parts for everything to be timely.</i>														
<i>Add in more soundtracks and art to enhance the experience of using the product.</i>														
<i>Show progress to the client and volunteer beta-testers, and get feedback on what is done so far.</i>														
<i>Make final changes to project as instructed by the client</i>														
<i>Complete Documentation & Submit</i>														

Analysis Phase

Pseudocode

LOAD Landing Page

If HOME clicked:

LOAD Home Page

[Redirect Here when Home Page Loaded]

If DEMO_QUIZ clicked:

LOAD Quiz Page

Set Topic to "Demo"

SHOW Demo

If Logged in:

SHOW Overview

If AccType = "Student":

SHOW Assigned_Pending_Quizzes

SHOW Accessible_Quizzes

If Do_Quiz clicked:

LOAD Quiz Page

Set Topic to [topic]

SHOW Question [Redirect here for each new question]

If SUBMIT clicked:

Check_Answer[]

SEND Score to DB

If Final_Question:

SEND Final_Score to DB

LOAD Home Page [Redirects above]

Else:

SHOW Next_Question [Redirects above]

Elif AccType = "Teacher":

SHOW Student_List

For Student in Student_List:

SHOW Assigned_Quizzes

SHOW Final_Score

If LOGIN clicked:

LOAD Login Page [Redirect Here when Login Page Loaded]

If SIGN_UP clicked:

SHOW Sign_Up Fields

If SUBMIT clicked:

If Entered_Data_is_valid:

HASH Login_Data as Hashed_Login_Data

SEND Hashed_Login_Data to DB

LOAD Home Page [Redirects Above]

If SUBMIT clicked:

LOAD Home Page

[Redirects Above]

If ABOUT_US clicked:

LOAD Contact Page

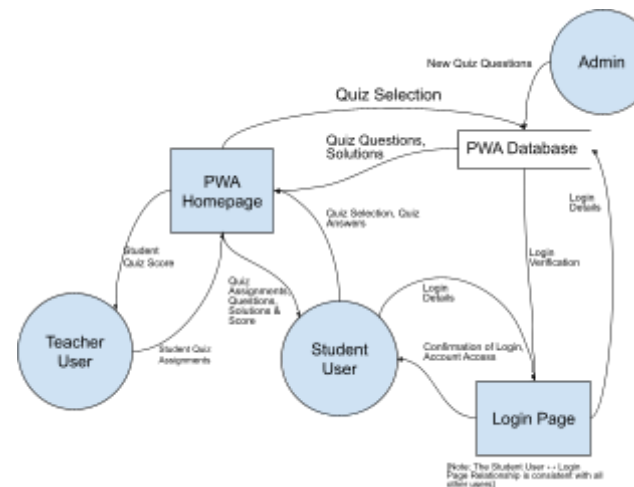
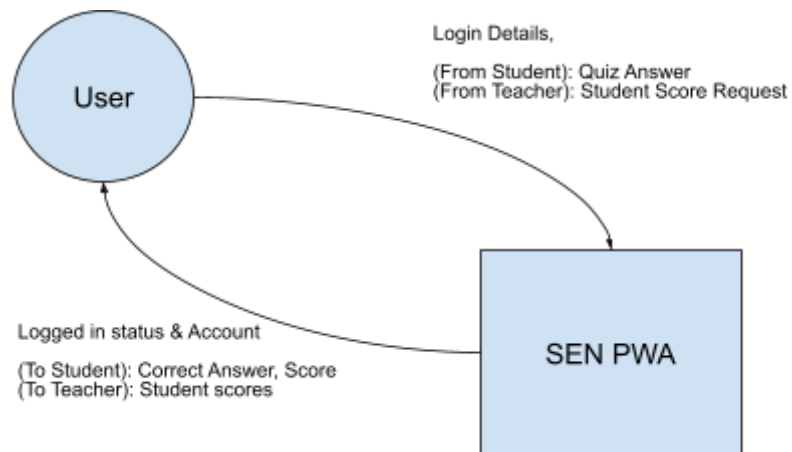
[Redirect Here when Contact Page Loaded]

SHOW Contact_Details

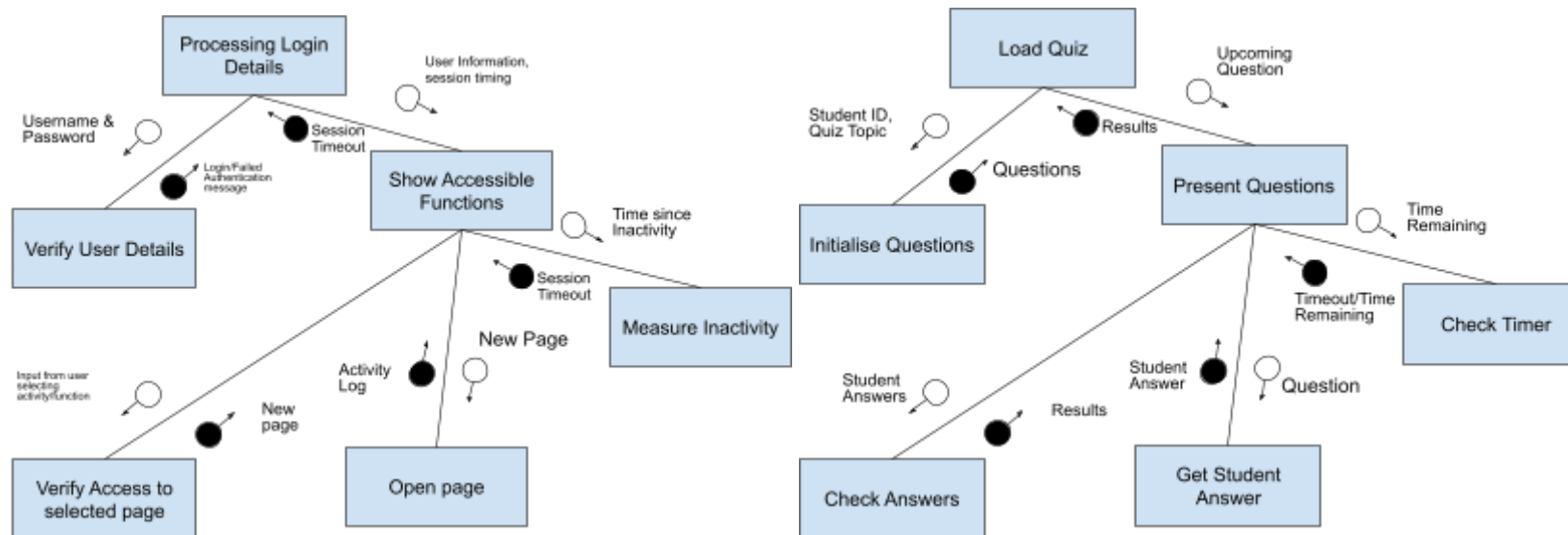
Context Diagram

&

Data Flow Diagram



Structure Chart (Login & Quiz)



Creating the Database

There will be multiple tables in the database based on question types, and some questions will have additional parameters within their tables.

The common attributes are as follows:

- QuestionID — Primary Key
- Question_Subtopic — String [what subtopic the question addresses or falls under]
- Question_Difficulty — Number [Is represented by a number out of 10]
- Question_Question — String [This is the actual question that will be asked to the student.]

The Unique attributes are as follows:

- Multiple Choice Questions
 - Option_A
 - Option_B
 - Option_C
 - Option_D
 - Correct_Option
- Match the Following
 - L1 — String
 - L2 — String
 - L3 — String
 - L4 — String [These are the 4 options on the left]
 - R1 — String
 - R2 — String
 - R3 — String
 - R4 — String [These are the 4 options on the right]
 - Correct_Combination — String [This shows the correct combination in the format of "L1R?L2R?L3R?L4R?"]
- Arrange
 - Item1 — String
 - Item2 — String
 - Item3 — String
 - Item4 — String
 - Correct_Arrangement — String [This will be in the format of "1234" or any respelling with the numbers 1-4.]
- Select All That Apply
 - Item1 — String
 - Item2 — String

- Item3 — String
- Item4 — String
- Correct_Items — String [This will have the correct options in ascending order of when the options appear, e.g. "23" or "124"]
- Number
 - Correct_Num — Number
- String
 - Correct_String — String

The next one will be for the signs or information blocks

- Sign
 - Sign_ID — Primary Key
 - Sign_Title — String [Used to Describe what the sign is about]
 - Sign_Subtext — String [This includes the paragraphs of text that is to be put out]
 - Sign_Image — Link [Link to the image will be provided].
 - Sign_Subtopic — String [What subtopic the sign addresses, used to locate where it should show up]

The next table is for the users

- All Users
 - User_ID — Primary Key [This is used to identify the users]
 - Name — String
 - Role — String [Could be Student, Teacher, Admin or Host]
 - Email — String
- Student

- Teacher — Number [the ID of the teacher]
- Assigned_Quizzes — String [Lists all the titles of the quizzes the student must complete.]
- Teacher
 - Students_ID — String [this will be given by converting the ID numbers of each Student and separating them by non-letter characters such as spaces or full stops. Can be empty if they have no students.]

Note: The Host and Admins [after inheriting roles from Teacher] do not have any additional roles.

Design Phase

Canva Designs for Application: <https://www.canva.com/design/DAG6mEq5Wk/QeS4X1T5EAhFi1n8a0QqZw/edit?ui=e30>

Progression within the App

- New accounts do not necessarily need verification so far to do the demo quiz. Only for the other quizzes will users be required to be logged in with a valid email address linked to their account.

Signs & Quizzes in order:

Demo:

1. Sign — Welcome to SEN. Tells you about how the website works I guess...
2. Sign — Tells you to prepare for the demo test, which is a 10 mcq pre-test.
3. Pre-Test — Algorithmic Thinking. 10 easy basic python operations questions [using +, -, * and /]. Not actually assigned and can be done by anyone, even those logged out. [Note: this was changed to be the SD Phases in the final product.]

4. Sign — Demo Finish. Tells the user to log in or sign up to do more quizzes.

SEN Thunder Mantles:

5. Sign — Asks you if you want to do a pre-test. If yes, it tells you to be ready for the pre-test, telling you to answer to the best of your ability.
6. Pre-Test — SD Phases. 10 questions regarding the waterfall approach.
7. Sign — Summary on the first few SD Phases
8. Sign — Summary on the next few SD Phases
9. Sign — Summary on the last few SD Phases
10. Test — SD Phases.

Development Phase

The Database

- The code on the right shows the initialisation of the database that the PWA connects to.
- The database uses sqlite3 and a cursor to connect and interact with the database.
- For the User table:
 - The username and password fields are used for authentication.
 - The email field is only used for the initial authentication and assigning the 'accesslevel' when an account is created.
- For the Question table:
 - The fields from q subtopic to question type refer to the respectively named aspects of the question.
 - The qitems refer to the items in four-option questions, and the ones with 'r' suffixes refer to the answering options in match questions.
 - Also recently added is an identifier for the use of specific functions to find the answer to specific questions with randomisers.
- For the Marks table:
 - There is an id for the one taking the test, the teacher that governs the test taker's account and an identifying number for the quiz.
 - There is also a field to track the score achieved by the test taker.
- For the Signs table:
 - Signs have unique titles to help identify them.
 - They also have text content and can have an image as well.

```
def get_db():
    return sqlite3.connect(DB_PATH)

def init_db():
    conn = get_db()
    cursor = conn.cursor()

    cursor.execute('''
        CREATE TABLE IF NOT EXISTS User (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL,
            password TEXT NOT NULL,
            email TEXT NOT NULL,
            accesslevel TEXT NOT NULL DEFAULT 'student'
        )
    ''')

    cursor.execute('''
        CREATE TABLE IF NOT EXISTS Question (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            qsubtopic TEXT NOT NULL,
            qinfotext TEXT,
            qquestiontext TEXT NOT NULL,
            qassociatedimage TEXT,
            qquestiontype INTEGER NOT NULL,

            qcorrectanswer TEXT NOT NULL,
            qitem1 TEXT,
            qitem2 TEXT,
            qitem3 TEXT,
            qitem4 TEXT,
            qitem1r TEXT,
            qitem2r TEXT,
            qitem3r TEXT,
            qitem4r TEXT
        )
    ''')

    cursor.execute('''
        CREATE TABLE IF NOT EXISTS Marks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            testtakerid INTEGER,
            teacherid INTEGER,
            quiznumber INTEGER,
            score INTEGER
        )
    ''')
```

The Flask App Routing

```
@app.before_request
def session_timeout(): #setting up a limit so that session times out after 2 hours of inactivity...

@app.route('/')
def home():
    init_sessionData()
    return render_template('index.html')
@app.route('/register', methods=['GET', 'POST'])
def register(): ...
@app.route('/login', methods=['GET', 'POST'])
def login(): ...
@app.route('/logout')
def logout(): ...
```

- The app runs on Flask, and the url endings can be used to navigate throughout the website. E.g. If there is no special ending, it goes to the home page, if the url ends in /login, it goes to the login page, etc.
- For some specific routes such as /checkanswer and /next_question, where there is no dedicated html document for the route, the app performs a certain function and then either renders a template on its own.

Account Features (Login & Registration)

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        uname = request.form['username']
        pwd = request.form['password']

        conn = get_db()
        cursor = conn.cursor()
        try:
            user = cursor.execute('SELECT id, password FROM User WHERE username = ?', (uname,)).fetchone()
            if user and bcrypt.checkpw(pwd.encode(), user[1] if isinstance(user[1], bytes) else user[1].encode()):
                session['user_id'] = user[0]
                session['incorrectAttempts'] = 0
                return redirect(url_for('loggedinhompage', loggedIn = True))
            else:
                try:
                    session['incorrectAttempts'] += 1
                    if session['incorrectAttempts'] >= 5:
                        timePenalty = (session['incorrectAttempts'] - 4) * 30
                        session['lastFail'] = datetime.datetime.now(tz=utc_plus_10)
                        if (datetime.datetime.now(tz=utc_plus_10) - session.get('lastFail')).total_seconds() < timePenalty:
                            return render_template('login.html', loginTimeout = (datetime.datetime.now(tz=utc_plus_10) - session['lastFail']).total_seconds())
                        else:
                            return render_template('login.html', error=f"Invalid username or password! Note that both fields are case-sensitive. {session['incorrectAttempts']}/5 incorrect attempts.")
                    else:
                        return render_template('login.html', error=f"Invalid username or password! Note that both fields are case-sensitive. {session['incorrectAttempts']}/5 incorrect attempts.")
                except Exception as e:
                    session['incorrectAttempts'] = 1
                    return render_template('login.html', error=f"Invalid username or password! Note that both fields are case-sensitive. {session['incorrectAttempts']}/5 incorrect attempts.")
        except sqlite3.Error as e:
            return render_template('login.html', error="An error occurred. Please try again.")
        finally:
            conn.close()
    if session.get('lastFail') and session.get('incorrectAttempts'):
        if session['incorrectAttempts'] >= 5:
            timePenalty = (session['incorrectAttempts'] - 4) * 30
            if (datetime.datetime.now(tz=utc_plus_10) - session.get('lastFail')).total_seconds() < timePenalty:
                return render_template('login.html', loginTimeout = round(timePenalty - (datetime.datetime.now(tz=utc_plus_10) - session['lastFail']).total_seconds()))
            else:
                return render_template('login.html')
    return render_template('login.html')

@app.route('/logout')
def logout():
    session.clear()
    init_sessionData()
    return redirect(url_for('login'))
```

- The login form checks inputted usernames and passwords against the hashed versions stored in the database, and if a match is successful, the app loads the logged in homepage. Otherwise, it will show that an incorrect attempt was made.
- The code shows that after every incorrect attempt following the 5th wrong guess, the app automatically times out the user for a time period that increments by 30 seconds for every failed attempt after the 5th one.
- The code also, on the front end, automatically loads the timer to update every second.
- When the user logs out, it clears the session data before re-initialising the critical session variables.

Data Dictionary

User Table in Database

Name	Datatype[s]	Description
id	Integer [Primary Key]	A unique identifier within the database for the user.
username	String	A unique text to identify the user by when logging in
password	String	A hashed password using bcrypt
email	String	Another unique string used only as a means to determine the access level of the user. This field is never accessed again unless to check if another account is being made under that name.
accesslevel	String	Used to determine the access level of a user [e.g. "student", "teacher"]
teachercode	String	The selfcode of the teacher a student's account is linked to. This attribute only applies to users with an access level of 'student'. Many students can have the same teachercode.

selfcode	String	A unique and more randomised identifier for a teacher that other students would use to connect their accounts to a teacher. This attribute only applies to users with an access level of 'teacher' or above. Teachers and selfcodes have a one-to-one relationship.
----------	--------	---

Question Table in Database

Name	Datatype[s]	Description
id	Integer [Primary Key]	A unique identifier within the database for the question.
qsubtopic	String	The subtopic a question belongs to.
qinfotext	String	Any associated 'info text' some questions may have that is referenced by the question.
qquestiontext	String	The actual question that the test taker can be asked.
qassociatedimage	String	The file name of the image that a question may refer to.
qquestiontype	Integer	In other instances referred to as the 'robotType'. Follows a key to determine what the format of the question is.
qcorrectanswer	String	What the inputted answer from a student, be it by selecting a multiple choice answer or directly inputting into a text box, is to be compared with to check if the student is correct or not.
qanswerspecialmethod	String	A key used to determine what the correct answer to a question that may have different answers based on

item1, item2, item3, item4	String	The four options in four option questions such as the multiple choice, select all that apply and arrange questions, as well as the match questions.
item1r, item2r, item3r, item4r	String	The 4 right options exclusively for the match questions.
difficulty	Integer	A number from 1-10 that helps decide how long the user would have to answer this specific question.

QuizResult Table in Database

Name	Datatype[s]	Description
id	Integer [Primary Key]	A unique identifier within the database for the result.
user_id	Integer [Foreign Key]	The id of the User that is associated with this result. A user can have many quizresults, but a quizresult can only refer to one user.
question_id	Integer [Foreign Key]	The id of the Question that this result came from. A question can have many quizresults, but a quizresult can only refer to one question.
quiz_index	Integer	The position of the question in the quiz.
given_answer	String	What answer the User gave.
correct_answer	String	The correct answer for this question.
is_correct	Integer	A number representing whether the user's answer was correct [1] or not [0].
timestamp	String	When this result was made in an isoformat datetime format.

Marks Table in Database

Name	Datatype[s]	Description
id	Integer [Primary Key]	A unique identifier within the database for the marksheet.
testtakerid	Integer [Foreign Key]	The id of the user [student] that took this test.
teacherid	Integer [Foreign Key]	The teacher associated with the student user that took this test.
quiznumber	Integer	A numerical identifier for the quiz.
score	Integer	The individual marks a student scored for the test.
totalmarks	Integer	What the test was out of in total.
subtopic	String	The subtopic that the test was on.

Signs

Name	Datatype[s]	Description
id	Integer [Primary Key]	A unique identifier within the database for the sign.
signtitle	String	A unique title for a specific sign.
signimageurl	String	The file name of any associated image for the sign.
signtext	String	The contents of the sign. Laced with <p> and <h2> tags that appear to further format the contents when presented in the sign.

AssignedCourses Table in Database

Name	Datatype[s]	Description
id	Integer [Primary Key]	A unique identifier within the database for the assignment.
student_id	Integer [Foreign Key]	What student the assignment is given to.
subtopic	String	The subtopic that the test that the student must do is on. A constraint is also in place to prevent a student from being assigned the same subtopic multiple times.
assigned_by	Integer [Foreign Key]	The id of the teacher that assigned the test.
assigned_at	String	The isoformat datetime at which the test was assigned.

Session Variables

Name	Datatype[s]	Description
session	Flask session	Stores the temporary data that is specific to the current user on the app.
user_id	Integer	The ID of the user if they are logged in.
incorrectAttempts	Integer	The number of incorrect guesses when logging in
lastFail	Datetime	The last time the user failed a login attempt
current_question_index	Integer	The current index within the testOrder [including signs]
question_number_within_	Integer	Same as above but excluding signs.

test		
test_order	List [dictionary]	Includes all the details for the signs and the basis of questions in a set order for a test.
current_question_data	Dictionary	Contains the details of the current question presented on the test page.
current_question_params	List [String]	The stored parameters used when generating the question.
quiz_score	Integer	A counter for what the user has gotten right so far.
quiz_total	Integer	The total number of questions/marks to get in the quizzes.
quiz_subtopic	String	The subtopic of the current quiz being taken.
qtimerstart	Datetime	Stores when the question was first loaded, and is used by the question timer in the test page.
qtimer_question_id	Integer	Used to identify the question and figure out when to store the timer data.

Other Variables used within the python script.

Name	Datatype[s]	Description
e	Error/Exception	The variable that stores the specific exception or error that was thrown within a try statement.
i	Integer	Used for indexing in for loops.
app	Flask object	This is the instance of the whole web application itself.

DB_PATH	String	The SQL database file location
COURSE_SUBTOPICS	String	A list containing all of the subtopics, which can be used throughout various parts of the application, including the assigning of courses and within question parameters.
utc_plus_10	Timezone [Datetime]	The time zone used for calculating times for everything in the app that uses a timer.
senPhases, eg_things_done_in_sen _phases, egstrings, glossaryTerms, examplesOfTerms	List [string], List [dictionary]	Used to globally store the question parameters for easy referral and use for finding answers.

Methods/Functions in app.py

Name	Description
auto_github_commit()	Commits the project to the respective repository
get_db()	Connects to the database
column_exists()	Checks if a column exists in a specific table within the database. This is only used when a new column has been recently added to the table following a new update.
add_column_if_missing()	It uses the above function to check if a column is missing, then adds the column if it is.
generate_unique_teacher_code()	Generates a random code that is different to the codes obtained by every other teacher account.
ensure_teacher_self_codes()	Creates a selfcode for any and all teachers in the database without a code when called.

init_db()	Creates the database and tables if they do not already exist.
purge_db()	Clears the database [primarily used in testing].
createDummyUser()	Creates the dummy users for exampleuser1 & exampleuser2.
init_sessionData()	Initialises the data for 'session' when connecting to the app.
generate_question_parameters()	Selects a random value from specific parameters when a question is being generated.
check_var_for_indicator()	Checks for indicators in questions such as "///randominteger///" in any part of a question, replaces it using the above function, and returns the same variable with the indicators swapped with parameters, which are also stored within the session.
get_sign()	Finds a sign by its title.
solve_question()	Uses the answer method outlined in very specific questions to solve each of them, based on the parameters that were used to build the questions.
generate_question_by_id()	Builds a question to be added to the session by finding all the questions that fit its constraints, fixing wherever there are indicators to include extra parameters and finding a solution using the above function for questions without a defined answer.
makeTestOrder()	Designs the whole test order in terms of the test pages for questions and the positioning of signs and the different types of questions throughout.
makeDemoTest()	Uses the above function and drafts a demo test, which is of a consistent format for all signed out users.
get_username_from_id()	Inputs a user id, and outputs the username as a string.

<code>convert_seconds_to_format()</code>	Takes in an integer value for how many seconds should be displayed on the timer and outputs a string in the form of minutes:seconds.
<code>is_logged_in()</code>	Checks if the user is logged in
<code>get_assigned_courses()</code>	Obtains the courses that are assigned to a student by inputting their id.
<code>assign_course_to_student()</code>	Adds a record to the database which shows a new course being assigned to a student user.
<code>remove_assigned_course()</code>	Mostly called upon the completion of an assigned quiz. Removes the record added in the above function.
<code>get_teacher_for_student()</code>	Uses the teacher code to see if a teacher with that code exists, and if they do, it outputs their details such as id, username and their code.

App Routes

Name	Description
<code>/ [home]</code>	The root URL, which goes to the logged out home page that has basic information about the site.
<code>/register</code>	The route used for either registering a new user, or if nothing is being posted, loads the register.html file.
<code>/login</code>	The route that logs the user in based on the credentials entered. If nothing was posted, it just loads the login page, login.html.
<code>/logout</code>	Pops the user id from the session and re-navigates back to the root URL.

/loggedinhomepage	Shows the homepage for when a user successfully logs into their account.
/check_inactivity	Applies a timer to 'session' so it expires when a user is not using the app for a long time.
/preparation	Loads the preparation.html page and if any subtopic is specified, it is shown on the screen before a course is started.
/dashboard	Loads the dashboard for an individual signed-in account.
/adminzone	If the logged in user is the host, they can access the page to add questions and/or signs to the database, which will then be used by the website users. If authorised, it loads adminzone.html.
/add_q_to_db	After taking inputs from the page loaded in the previous route, authenticates the host before adding the question into the database.
/add_s_to_db	The same as the above but logs in a sign instead.
/assign_teacher	Checks for a teacher with a specified selfcode and then assigns it as the teachercode for a student.
/assign_course	Adds a course on the student's to-do list.
/get_student_scores	Gets the scores a specific student received on their quizzes and compiles them into a json format to be used later.
/signpage	Loads the sign page, signpage.html, with appropriate text being shown using JINJA.
/testpage	Similar to the above one, but it loads testpage.html, which is slightly more interactive and functionally important.
/checkanswer	Checks the answer inputted into the test page and loads an appropriate page and the result of

	the question.
/start_test	Creates the test that will be completed using the makeTestOrder() function.
/next_question	Loads the next question or sign based on the test order.
/qtimerexpired	When called upon timeout, provides the answer to the question.

Integration Phase

Running the Server Locally

- When testing the app is run on a local server via the IDE of Visual Studio Code, which can be accessed by clicking a link shown in the terminal which can have its URL posted into a browser window and viewed. [Top]

- This means that the terminal will show which app routes are being called and which static files are being accessed. [Bottom]

```
if __name__ == '__main__':
    with app.app_context():
        init_db()
        createDummyUser()

    app.run(debug=True)
```

```
127.0.0.1 - - [20/Jun/2026 21:16:19] "POST /checkanswer HTTP/1.1" 200 -
127.0.0.1 - - [20/Jun/2026 21:16:19] "GET /static/manifest.json HTTP/1.1" 200 -
127.0.0.1 - - [20/Jun/2026 21:16:19] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [20/Jun/2026 21:16:19] "GET /static/swampbg.png HTTP/1.1" 200 -
```

The HTML Pages

- The html documents are all linked with style.css as their stylesheet. [Top]
- The pages are also linked to app.py via the app routes [Bottom]
- Within the routes, the html pages are projected onto the web browser using the function 'render_template()', with its initial parameter being the html file name, followed by any other parameters that need to be passed through.
- This works through the html documents being in the 'templates' directory.

```
<link rel="stylesheet" type="text/css" href="{{ url_for('static', filename='style.css') }}">
```

```
@app.route('/')
def home(): ...

@app.route('/register', methods=['GET', 'POST'])
def register(): ...

@app.route('/login', methods=['GET', 'POST'])
def login(): ...

@app.route('/logout')
def logout(): ...

@app.route('/loggedinhomepage')
def loggedinhomepage(): ...
```

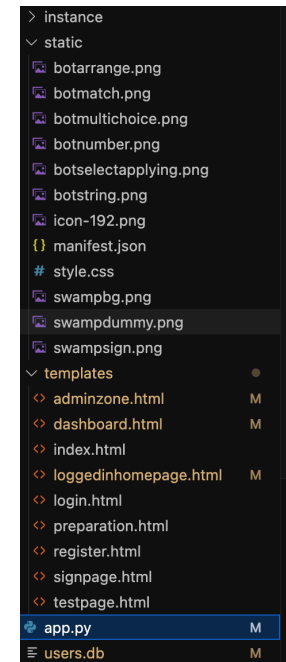
Images/Icons & Stylesheet

- Images/icons, alongside style.css, exist inside a folder called 'static', which includes the files that will not change for different users, and thus can be stored and used to load parts of the app without needing to use the server to dynamically generate the files.

```
def get_db():
    return sqlite3.connect(DB_PATH)
def column_exists(cursor, table, column):
def add_column_if_missing(cursor, table, column, definition):
def generate_unique_teacher_code(cursor):
def ensure_teacher_selfcodes(cursor):

def init_db():
    conn = get_db()
    cursor = conn.cursor()

    cursor.execute('''
        CREATE TABLE IF NOT EXISTS User (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL UNIQUE,
            password TEXT NOT NULL,
            email TEXT NOT NULL UNIQUE,
            accesslevel TEXT NOT NULL DEFAULT 'student',
            teachercode TEXT,
            selfcode TEXT UNIQUE
        )
    ''')
```



The Database

- Connects with the application via the sqlite3 cursor.
- The connection via the cursor uses SQL to contact the database and access/modify its contents.

Testing & Debugging Phase

Testing Cycles

Test Cycle Date	What was tested	Issues	Changes Made
7/6/2026	• The ability to use the login form by checking username and password • The incorrect attempt checker. • The timeout feature.	• The incorrect attempt tracking feature works just fine, as well as the logging in using an existing login. • There is about a second where a different number shows up when it gives a number such as 1.0e8 • There was a period where the program gave an error saying it could not subtract a naive and aware datetime object.	• Added a timezone specification in the parameter for tracking the time using datetime to prevent the naive/aware error. • Tried to make it reload immediately so the absurd number would not be visible. This does not always work.
14/6/2026	• The transition from list storage to dictionary storage for signs and questions for	• There were frequent errors and crashes. • There were also frequent	• Followed through individual lines of code to figure out where things were going so

	ease of access purposes. • Test the test building capabilities on a demo quiz.	overriding of some lines of code causing the index to count the position within the quiz a test taker is at to increment twice and miss questions. • The lists were generating the structures as expected, however.	wrong all the time. • As a temporary fix, added some code that would decrease the index to get things moving in the correct order again.
15/6/2026	• Tested the option generation and answer checking abilities of the multiple choice questions • Made sure the site could handle using some of the newer logics added.	• There were frequent KeyErrors • Sometimes the options repeated. • The correct answer was always option A, and sometimes others if they had the same text as option A.	• Turns out there was a rogue bit of code that was piggybacking off the wrong dictionary key, and was the reason for so many key errors and once that was fixed, everything seemed to run smoothly. • Implemented a random.shuffle algorithm to shuffle the options when generating the options for multiple choice.
16/6/2026	• Once again tested the testing functionality using the same dataset. • Tested the demo test for	• The incrementing of questions went up by 2, meaning a certain function for the next question was	• Found that it was a line of code catching an exception that executed another function multiple times.

	functionality	being called twice. The timer was not counting down as expected.	- Changed the fetch statement in the button for going to the next question into a form that would submit when pressing the button, and this somehow ended up fixing the double counting glitch. - Fixed the script to update the timer similar to the login timeout feature.
19/6/2026	Tested the ability to assign teachers to student accounts	- The app did not recognise the url when a teacher code was sent to be matched with a teacher. - The feature does not show whether the student has completed the assigned modules or not.	Fixed the assign_module route so it did end up redirecting to the correct route when called.
23/6/2026	The integration of the host role, which can add questions and signs that can be used in other question templates.	There was some slight improper rendering of the buttons, and forms.	- Added small improvements to the UI to make it look smoother. - Put the host user as a dummy variable.

Installation Phase

- The Instructions on how to operate the app is on the README file in the git repository.
- The application requires an IDE such as Visual Studio Code to set up the hosting of the website.
- When the website is globally hosted, it runs on a seemingly randomly generated IP address, which can be inputted as a http link, and this way, anyone can access the site.

Maintenance Phase

Company Statement

As the CEO and sole employee of Light Guard Studios, I, Skanda Nagendra, on behalf of the company, would like to provide assurance that the product, the web app that goes by the name of 'SEN', will be continuously maintained over time with frequent updates to security, infrastructure and other features to keep improving the app.

As there will always be many different sorts of questions that can come up in Software Engineering Exams, it is good for students to be up-to-date and with continued exposure to some of the kinds of questions that can be asked and so with each upcoming update, the app will have its questions list updated alongside the database.

The app will also be continuously improved in the future to include more topics, subtopics, and potentially a wider syllabus worth of content.

END-USER LICENSE AGREEMENT (EULA)

SEN — Light Guard Studios

IMPORTANT — READ CAREFULLY:

This End-User License Agreement ["EULA"] is a legal agreement between you [an individual or a single entity] and Light Guard Studios ["Licensor"] for the 'SEN' software, including any associated media, printed materials, and "online" or electronic documentation ["Software"]. By installing, copying, or otherwise using the Software, you agree to be bound by the terms of this EULA. If you do not agree to the terms of this EULA, do not install or use the Software.

1. Grant of License

Licensor grants you a non-exclusive, non-transferable, limited license to install and use one copy of the Software on a single computer for personal, non-commercial, educational, and development purposes.

2. Ownership

The Software is owned by Licensor and is protected by copyright laws and international treaty provisions. Licensor retains all rights, title, and interest in and to the Software, including all copyrights, trademarks, trade secrets, and other intellectual property rights. You acknowledge that you obtain no ownership interest in the Software through your use of it.

3. Restrictions

You may not:

- a. Modify, translate, reverse engineer, decompile, or disassemble the Software.
- b. Create derivative works based on the Software.
- c. Rent, lease, lend, sell, redistribute, or sublicense the Software.
- d. Remove or alter any copyright notices or other proprietary markings on the Software.
- e. Use the Software for any commercial purposes, including but not limited to selling the game, using it for commercial demonstrations, or charging for access to it, without express written permission from the Licensor.
- f. Distribute or publish the Software on any platform or website without explicit permission from the licensor.

4. Educational and Development Use

This license is specifically granted for educational and development purposes. You may use the Software to learn game development, create portfolio pieces, and demonstrate your skills.

5. Termination

This EULA is effective until terminated. Licensor may terminate this EULA if you fail to comply with any term or condition of this EULA. Upon termination, you must destroy all copies of the Software.

6. Disclaimer of Warranty

THE SOFTWARE IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND. LICENSOR DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON INFRINGEMENT. THE ENTIRE RISK ARISING OUT OF USE OR PERFORMANCE OF THE SOFTWARE REMAINS WITH YOU.

7. Limitation of Liability

IN NO EVENT SHALL LICENSOR BE LIABLE FOR ANY DAMAGES WHATSOEVER (INCLUDING, WITHOUT LIMITATION, DAMAGES FOR LOSS OF BUSINESS PROFITS, BUSINESS INTERRUPTION, LOSS OF BUSINESS INFORMATION, OR ANY OTHER PECUNIARY LOSS) ARISING OUT OF THE USE OR INABILITY TO USE THE SOFTWARE, EVEN IF LICENSOR HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

8. Governing Law

This EULA shall be governed by and construed in accordance with the laws of NSW, Australia, without regard to its conflict of law principles.

9. Entire Agreement

This EULA constitutes the entire agreement between you and Licensor with respect to the Software and supersedes all prior or contemporaneous communications and proposals, whether oral or written, between you and Licensor with respect to the Software.

10. Contact Information

If you have any questions about this EULA, please contact:

skanda.nagendra@education.nsw.gov.au

By installing or using the Software, you acknowledge that you have read, understood, and agree to be bound by the terms and conditions of this EULA.

Appendix

Journal

Term 3 Holidays (28/09/2025 – 12/10/2025)

- Composed some soundtracks
 - These were named “Fading Grass”; “Approaching Greener Grass”; “An Ambient End”; “Look, Your Majesty!”; “Someone’s Coming...!” and “trucking along”, respectively.
- Looked at various inspirations for coming up with an idea for how the project should be structured.

17/10/2025

- Spoke with Mr Rose and confirmed him as the client
- Arranged to meet the next Tuesday

28/10/2025

- Met with Mr Rose in the Workshop and Discussed the project in detail, including all the dynamics

29/10/2025 - 19/12/2025

- Made some concept art and more music, and learned more in the background about building a PWA throughout the first term’s project.

20/12/2025 - 31/1/2026

- Made some designs on Canva that are yet to be converted into HTML for the webpage in certain aspects of the project.
- Made more songs that go by the names of "Artificial warmth" and "Whispers of Royal Favour". Modified some of the others to give them some variety.
- Created a system that would recognise questions under specific subtopics, which can be used to differentiate the questions and thus help make it easier to have certain questions in certain tests.
- Made separate tabs to work on the different aspects of documentation simultaneously with the project.

2/2/2026

- Wrote down more on the requirements and specifications and worked on some of the diagrams for the Analysis Phase.
- Copied some of the code from the guide to attempt to start building the PWA.

3/2/2026

- Copied more code from the guide and ended up having to modify some of it due to minor issues with certain characters.

5/2/2026 - 24/2/2026

- Worked throughout the documentation to try to finish the first few phases by week 6 Monday

25/2/2026

- The deadline has been pushed to Friday of Week 6.

28/2/2026 - 19/3/2026

- Worked on and off on the project before the half-yearly exam block.
- Focused on improving some of the designs and figuring out how to make the PWA suitable for use.
- Copied some of the code from the PWA tutorial on Google Classroom, exporting it to GitHub.

Post-Term 1 Holidays

- Was in India, but during that time, did manage to get through most of the instructions for the PWA.
- There were issues with the code and a lot of aspects of it simply did not work, and nothing ever loaded, despite all troubleshooting efforts.
- Abandoned it and restarted on a new GitHub Repository so there would be no clashing code.
- Made a separate app.py file to work on, after seeking out various tutorials.
- Made a basic index.html page so that there is something that can actually be seen.

24/4/2026 - 30/4/2026

- Integrated a basic stylesheet to ensure linking works.
- Designed and developed the basic parts of the app, with functionality being saved for later.
- Arranged another client meeting and discussed the project.

1/5/2026

- Linked the home, login and register pages.
- Made a login and register form.

2/5/2026 - 6/5/2026

- Debugged the login & registration forms to try and get them to allow new users to log in

7/5/2026

- Showed the teacher progress so far
- Found recurring issues with using SQLAlchemy for the database, so switched to sqlite3 and tried to remodel the code that had been done so far to fit sqlite3 instead.

8/5/2026 - 13/5/2026

- Continued debugging to see what the issues were with the sqlite3 model.
- Added some questions to the example question bank

14/5/2026 - 31/5/2026

- Worked on the other aspects of the app, including ways to navigate between the different pages
- Worked on making some more pages for the app, including the sign page, test page and a logged in homepage.
- Started building a set of questions that could be used in the app.

1/6/2026 - 4/6/2026

- The scratch project I had used to make my art for this project got corrupted and a significant amount of image assets that were going to be used no longer exist.
- Overhauled the UI design, as well as some specific interactions and functionality.

- Created some functions to initialise the signs and questions.

5/6/2026 - 7/6/2026

- Fixed the login form after a long period of trial and error, and also included a screen that updates a login timeout period
- Slightly changed other parts of the website to make it look better, visually.
- Tried to apply a similar method onto a timer feature for the test page.
- Used some manipulation within the syntax of the questions that are to be seeded to allow for questions to be customised and reused with random values.

8/6/2026 - 10/6/2026

- Started making some signs for the demo quiz, and using the current library to help create the progression from signs and questions in order, for the demo quiz.
- Made some more general bug fixes and smaller optimisations around the app and html pages.
- Worked on some more parts of the documentation.

11/6/2026

- Had another client meeting with Mr Rose to show him the progress on the PWA so far.
- Discussed adjusting the scope of the project to first get a working demo quiz over the weekend.

12/6/2026 - 15/6/2026

- Worked on the quiz and made a general code for the quiz to work through.

- The code works for two very specific kinds of multiple choice questions, which is a good sign that implies it could work for broader categories with a greater variety of questions.

16/6/2026 - 18/6/2026

- Adjusted the UI and moved appropriate style sections to the css stylesheet.
- Got the demo test fully functional
- Added more functionality such as the ability to assign and complete quizzes between teacher and student accounts.
- Added some new quiz questions to the database loader
- Finished the demo quiz and used basic templates for the quizzes on the other topics.

19/6/2026 - 21/6/2026

- Updated some of the text throughout some of the pages so they no longer just had placeholder text.
- Removed many redundancies that were found throughout mainly the app.py file.
- Added to the unfinished sections of the report.

22/6/2026 - 24/6/2026

- Worked on finishing the report section.
- Tested the global hosting capabilities outside of the local hosting.
- Integrated the host role, which is a modified version of the teacher role, which can update the questions and signs tables.
- Submitted the final project after committing everything to GitHub and handing in the link to the project.

Client Meeting Logs

1. 28/10/2025

<u>Date & Time of Meeting</u>	28/10/2025, ~1:35pm - 2:00pm
<u>Location of Meeting</u>	36T, Workshop.
<u>People Present</u>	S. Nagendra, D. Rose
<u>Apologies</u>	N/A

<u>Item</u>	<u>Description</u>	<u>Action</u>	<u>Person Responsible</u>	<u>Due Date</u>
1.	Know what needs to be coded, starting from one small topic.	Study the syllabus for Year 11 Software Engineering, and plan a program for just one specific topic.	Skanda Nagendra	None Specified
2.	Subject the user to a pre-test to gauge their knowledge before learning the content.	Design additional pages and have some sort of method to change the experience based on	Skanda Nagendra	None Specified

2. 30/4/2025

<u>Date & Time of Meeting</u>	30/4/2026 ~ 11:00 - 11:15
-----------------------------------	---------------------------

<u>Location of Meeting</u>	32T
<u>People Present</u>	S. Nagendra, D. Rose
<u>Apologies</u>	N/A

<u>Item</u>	<u>Description</u>	<u>Action</u>	<u>Person Responsible</u>	<u>Due Date</u>
1.	Have a working app to test for it to be tested and receive feedback on it.	Have a working App to test on Year 10 Design & Technology students, who will be the beta testers, and have them test specific aspects of the project.	Skanda Nagendra, David Rose	Term 2 Week 6
2.	Completion of the Application	Completion of the application after integrating feedback and receiving final evaluations.	Skanda Nagendra	Term 2 Week 8
3.	Pay attention to the scope of the project.	Define what 'success' would look like for the app, and plan effectively in the short term to complete everything within the time frame	Skanda Nagendra	None Specified

3. 11/6/2026

<u>Date & Time of Meeting</u>	11/6/2026 ~ 14:10 - 14:40
--	---------------------------

<u>Location of Meeting</u>	Near TAS Staffroom
<u>People Present</u>	S. Nagendra, D. Rose
<u>Apologies</u>	N/A

<u>Item</u>	<u>Description</u>	<u>Action</u>	<u>Person Responsible</u>	<u>Due Date</u>
1.	Implement the code for the questions	Implement the current logic and make the website functional for at least a short test of ~10 questions.	Skanda Nagendra	Friday 12/6/2026
2.	Adjust the Project Scope	Adjust the scope of the project to work for a small aspect of the first unit before proceeding to add more. Everything must be functional and able to be tested for feedback	Skanda Nagendra	Monday 15/6/2026

4. 19/6/2026

<u>Date & Time of Meeting</u>	11/6/2026 ~ 14:10 - 14:40
<u>Location of Meeting</u>	Near TAS Staffroom
<u>People Present</u>	S. Nagendra, D. Rose
<u>Apologies</u>	N/A

<u>Item</u>	<u>Description</u>	<u>Action</u>	<u>Person Responsible</u>	<u>Due Date</u>
1.	Put together a final product that is at least presentable to the class.	This should have all the planned features completed and ready to be tested.	Skanda Nagendra	Monday 22/6/2026
2.	Do not give away the answers in the quizzes immediately. When a student gets something wrong, they should be directed to wherever they can get additional information.	Add a link to additional information in some way. Considering the lack of time available to create special sections for individual questions or groups of questions, this may need to be left as a link to the syllabus.	Skanda Nagendra	Tuesday 23/6/2026
3.	Show finished product and get feedback before making final changes.	Show the finished project to the client and ensure it has been tested and feedback has been received on the product via a platform such as Google Forms, with appropriate changes made.	Skanda Nagendra	Tuesday 23/6/2026

Example Question Bank

Info Text	Question Text	Expected Answer	Subtopic	Difficulty
N/A	What step comes before/after ___ in the Waterfall Approach?	Whichever stage is what comes next. If multiple choice, the answer will be	SEN Fundamentals — Introducing SD Phases	3

		given among other options.		
N/A	In what stage of the Waterfall approach is ___?	A single word answer, it will be made lowercase before comparison is done. If multiple choice, the answer will be given among other options.	SEN Fundamentals — Introducing SD Phases	4
N/A	What datatype is ___?	Random assortment between strings, integers, floats [for decimals], and booleans. Always multiple choice.	SEN Fundamentals — Understanding Datatypes	2
<pre>x = {a} y = {b} sum = 'x' + 'y' print(sum)</pre>	What will the output of this code be?	The correct answer should be “xy”	SEN Fundamentals — Inputs and Outputs	3
<pre>x = {a} y = {b} sum = x + y print(sum)</pre>	What will the output of this code be?	The correct answer should be the actual sum	SEN Fundamentals — Inputs and Outputs	3
<pre>x = {a} y = {b} sum = x + y print('sum')</pre>	What will the output of this code be?	The correct answer should be ‘sum’s	SEN Fundamentals — Inputs and Outputs	3

<pre>x = {a} For i in range[{b}]: x += {c} print[x]</pre>	What will the output of this code be?	The actual code is run and calculated, then matched with the inputs. If this is multiple choice, there is a chance for an option to be some sort of error or None. $a, b, c \in \mathbb{Z}^+$	SEN Fundamentals — Inputs and Outputs	5
<pre>x = {a} For i in range[{b}]: x += {c}</pre>	What will the output of this code be?	The correct answer should be None. The question can be a number or a multiple choice question.	SEN Fundamentals — Inputs and Outputs	4
<pre>x = {a} For i in range[y]: x += {b} print[x]</pre>	What should the variable “y” be so that the output is __?	It should be, as an integer, $(x-a)/b$ The question can be a number or a multiple choice question.	SEN Fundamentals — Inputs and Outputs	5
<pre>x = {a} For i in range[{b}]: x += y print[x]</pre>	What should the variable “y” be so that the output is __?	It should be, as an integer, $(x-a)/b$ The question can be a number or a multiple choice question.	SEN Fundamentals — Inputs and Outputs	5
N/A	What is __ in binary? Numbers from 0-32	The app calculates the binary and converts the number output to a string, which is then matched with the input.	Mechatronics — Binary	3